

API Performance & Error Monitoring Report

Prepared By

Name: Gayatri kasbekar

Tools Used: Python, MySQL, Power BI

Connect With Me :-

[github](#)

[linkedin](#)

Index

1. Project Overview
2. Problem Statement
3. Objectives
4. Tools & Technologies Used
5. Dataset Description
6. Data Cleaning Process
7. Exploratory Data Analysis (EDA)
8. SQL Database Design & Analysis
9. Power BI Dashboard Development
10. Key Performance Indicators (KPIs)
11. Business Recommendations
12. Conclusion
13. Future Improvements

1. Project Overview

This project is an end-to-end data analytics solution designed to monitor and analyze API performance, user activity, and system reliability. The main objective is to identify performance bottlenecks, track error patterns, and optimize API efficiency using data-driven insights.

The project integrates multiple datasets including API details, user information, usage logs, and server performance metrics. The data is cleaned, transformed, and analyzed using Python and SQL, and visualized through an interactive dashboard built in Microsoft Power BI.

2. Problem Statement

Modern applications rely heavily on APIs to exchange data between systems, services, and users. As API usage grows, organizations face challenges in maintaining performance, reliability, and scalability.

The key problem is the lack of a centralized system to monitor and analyze API performance, user behavior, and infrastructure health in real time. This makes it difficult to identify slow-performing APIs, track error patterns, understand usage trends, and detect system bottlenecks.

Without proper analysis, issues such as high response time, increased failure rates, uneven server load, and inefficient resource utilization go unnoticed, leading to poor user experience and reduced system efficiency.

3. Objectives

The objective of this project is to build an end-to-end analytical solution using datasets related to APIs, users, usage logs, and server performance to:

- Monitor API performance and reliability
- Identify high-error and slow-performing APIs
- Analyze usage patterns across users and regions

- Detect infrastructure bottlenecks and system load issues
- Provide actionable insights for performance optimization

4. Tools & Technologies Used

This project was developed using a combination of data analysis, querying, and visualization tools:

- Pandas → Data cleaning, transformation, and analysis
- NumPy → Numerical operations and calculations
- Matplotlib → Basic data visualization
- Seaborn → Advanced visual analysis
- SQL → Data extraction, joins, CTEs, and window functions
- Microsoft Power BI → Interactive dashboards, KPIs, and reporting

5. Dataset Description

This project uses multiple interconnected datasets to analyze API usage, performance, and system behavior.

APIs Dataset

Contains information about available APIs and their metadata.

Key Columns:

- `api_id` → Unique identifier for each API
- `api_name` → Name of the API
- `api_category` → Category of API (Auth, Payments, Analytics, etc.)
- `status` → API status (Active, Deprecated, Testing, Inactive)
- `owner_team` → Team responsible for API

Users Dataset

Contains user and subscription-related information.

Key Columns:

- `user_id` → Unique user identifier
- `username` → User name
- `company_name` → Organization name
- `subscription_type` → Plan type (Free, Basic, Premium, Enterprise, etc.)
- `country` → User location
- `signup_date` → Account creation date

API Usage Dataset

Tracks API consumption and request activity.

Key Columns:

- `usage_id` → Unique usage record ID
- `user_id` → Links to Users table
- `api_id` → Links to APIs table
- `usage_date` → Date of API usage
- `total_requests` → Total API calls
- `successful_requests` → Successful calls
- `failed_requests` → Failed calls

API Logs Dataset

Captures detailed system performance metrics.

Key Columns:

- `log_id` → Unique log ID
- `api_id` → Links to APIs table
- `request_time` → Timestamp of request

- `response_time_ms` → API response time
- `status_code` → HTTP status code
- `request_method` → API method (GET, POST, PUT, DELETE, PATCH)
- `cpu_usage` → CPU utilization
- `memory_usage` → Memory utilization
- `request_count` → Number of requests processed
- `server_region` → Data center region
- `error_message` → Error details (if any)

Relationship Summary

- `user_id` → links Users and Usage datasets
- `api_id` → links APIs, Usage, and Logs datasets

6. Data Cleaning Process

- Identified and handled missing values by replacing with “Unknown” or removing critical null records.
- Removed duplicate records to ensure data accuracy and avoid analysis bias.
- Standardized text values (e.g., subscription type, country, status) by removing extra spaces and fixing inconsistent formatting.
- Converted string date columns into proper datetime format for time-based analysis.
- Filtered invalid values such as negative response times and incorrect request counts.
- Detected and treated outliers in numerical data using the IQR method.
- Corrected data types (string → numeric, string → datetime) for proper analysis.
- Merged multiple datasets using common keys like `user_id` and `api_id` to create a unified dataset.

- Ensured cleaned data was ready for analysis in SQL, Python, and Microsoft Power BI dashboards.

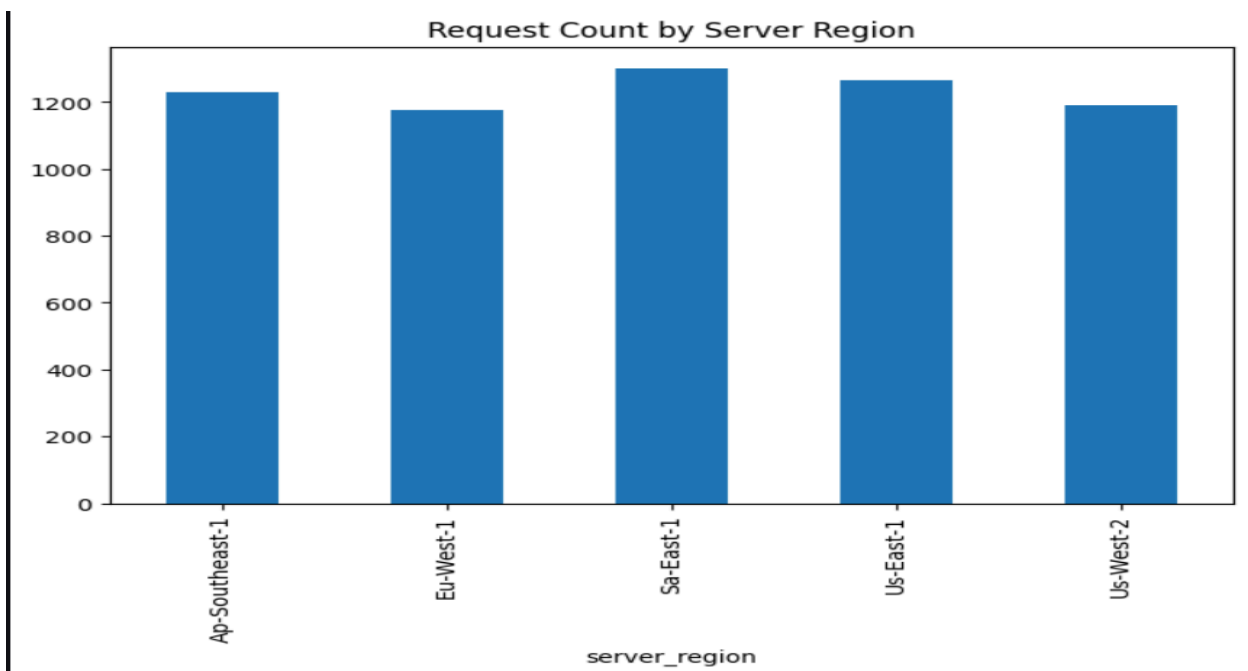
7. Exploratory Data Analysis (EDA)

EDA was performed using:

- Pandas
- NumPy
- Matplotlib
- Seaborn
-

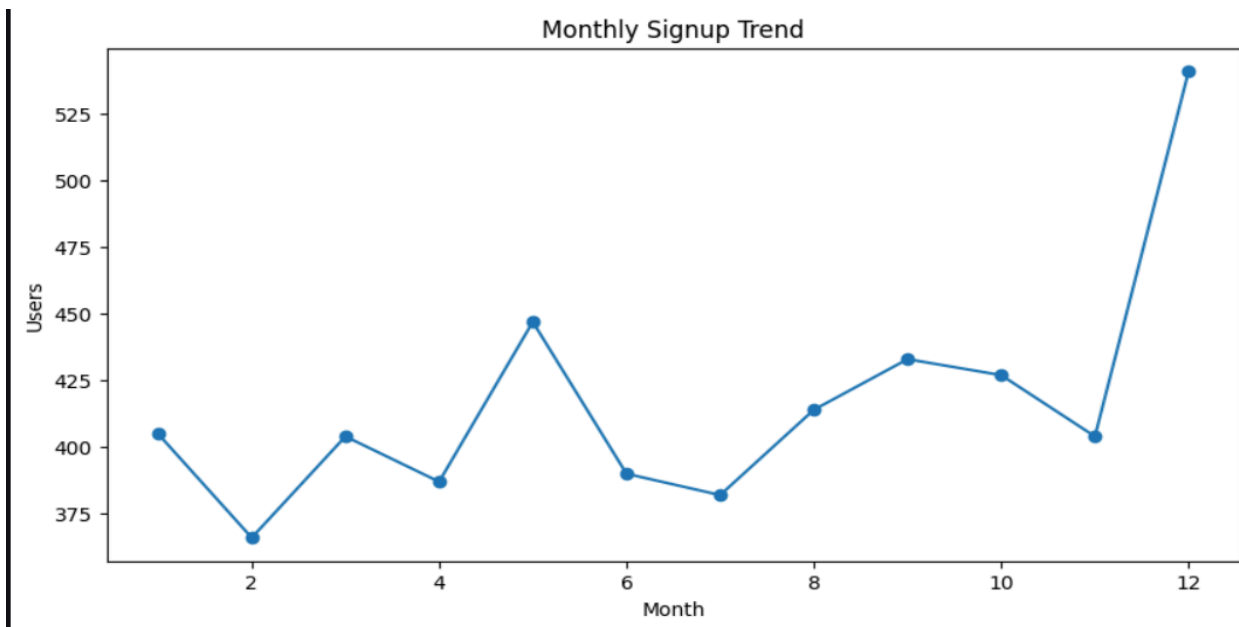
7.1 Request Count By Server Region

This bar chart shows the total number of incoming network or API requests distributed across five different geographical server regions. The workload is relatively well-balanced, with Sa-East-1 handling the highest traffic volume (around 1,300 requests) and Eu-West-1 handling the lowest.



7.2 Monthly SignUp Trend

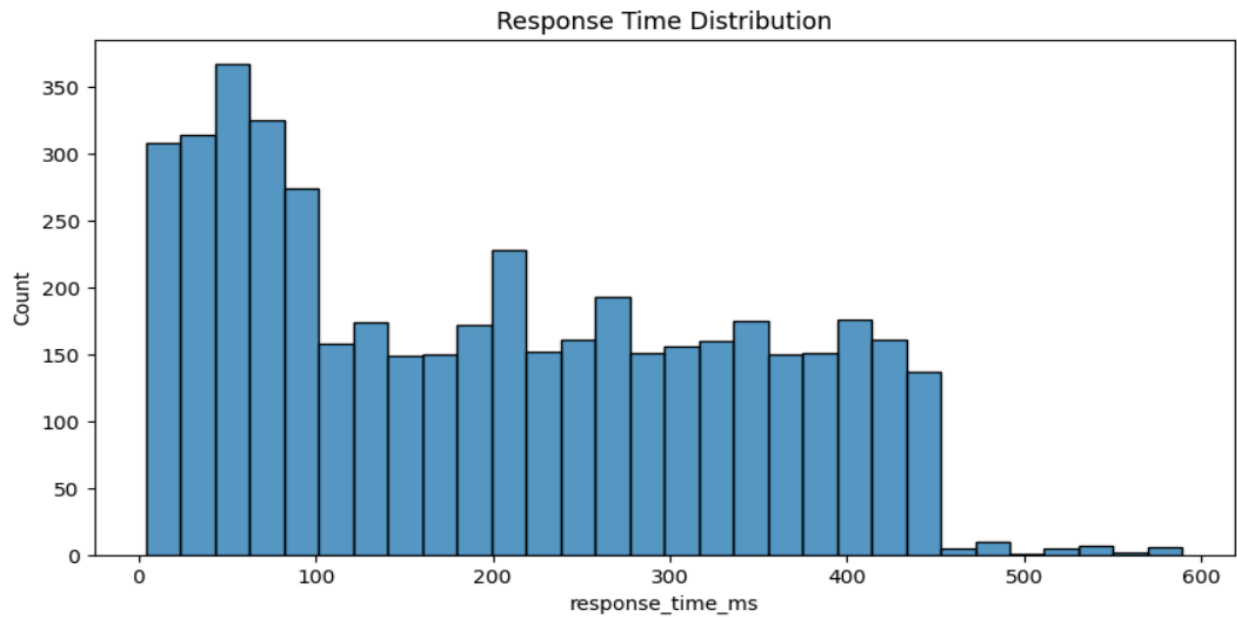
This line chart tracks user registration trends across a 12-month period. It shows that while signups fluctuate between roughly 360 and 450 users for most of the year, there is a massive surge in month 12, where signups peak at their highest level of over 540 users.



7.3 Response Time Distribution

This histogram illustrates the distribution of server response times in milliseconds (ms).

The majority of requests are highly optimized, peaking sharply between 50 ms and 100 ms. Following this initial spike, the distribution flattens into a relatively steady plateau ranging from 100 ms to 450 ms, before dropping off significantly with only a few outlier requests stretching beyond 500 ms.



8. SQL Database Design & Analysis

8.1 Database Schema

```
CREATE TABLE apis (  
  api_id INT PRIMARY KEY,  
  api_name VARCHAR(100),  
  api_category VARCHAR(50),  
  endpoint VARCHAR(255),  
  version VARCHAR(20),  
  owner_team VARCHAR(50),  
  status VARCHAR(20),  
  service_name VARCHAR(20)  
);  
  
CREATE TABLE api_logs (  
  log_id INT PRIMARY KEY,  
  api_id INT,  
  request_time DATETIME,  
  response_time_ms FLOAT,  
  status_code INT,  
  request_method VARCHAR(10),  
  payload_size_kb FLOAT,
```

```

    cpu_usage FLOAT,
    memory_usage FLOAT,
    request_count INT,
    error_message VARCHAR(255),
    server_region VARCHAR(50),
    is_error text,
    performance_category VARCHAR(20),

    FOREIGN KEY (api_id) REFERENCES apis(api_id)
);

```

```

CREATE TABLE users (
    user_id INT PRIMARY KEY,
    username VARCHAR(100),
    company_name VARCHAR(100),
    subscription_type VARCHAR(50),
    country VARCHAR(50),
    signup_date DATE
);

```

```

CREATE TABLE api_usage (
    usage_id INT PRIMARY KEY,
    user_id INT,
    api_id INT,
    usage_date DATE,
    total_requests INT,
    successful_requests INT,
    failed_requests INT,
    success_rate float,
    error_rate float,
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (api_id) REFERENCES apis(api_id)
);

```

8.2 SQL Analysis Queries

-- Rank APIs by Total Requests

```

WITH api_usage_rank AS (
    SELECT a.api_name, SUM(u.total_requests) AS total_requests,
           RANK() OVER(ORDER BY SUM(u.total_requests) DESC) AS rnk

```

```
FROM api_usage u
JOIN apis a
ON u.api_id = a.api_id
GROUP BY a.api_name
)
```

```
SELECT *
FROM api_usage_rank
WHERE rnk <= 10;
```

-- Running Total of Monthly Requests

```
WITH monthly_requests AS (
    SELECT MONTH(usage_date) AS month_no, SUM(total_requests) AS monthly_total
    FROM api_usage
    GROUP BY MONTH(usage_date)
)
SELECT month_no, monthly_total,
SUM(monthly_total) OVER(ORDER BY month_no) AS running_total
FROM monthly_requests;
```

-- APIs Performing Above Average

```
WITH api_requests AS (
    SELECT a.api_name, SUM(u.total_requests) AS total_requests
    FROM api_usage u
    JOIN apis a
    ON u.api_id = a.api_id
    GROUP BY a.api_name
)
SELECT api_name, total_requests
FROM api_requests
WHERE total_requests >
(
    SELECT AVG(total_requests)
    FROM api_requests
)
ORDER BY total_requests DESC;
```

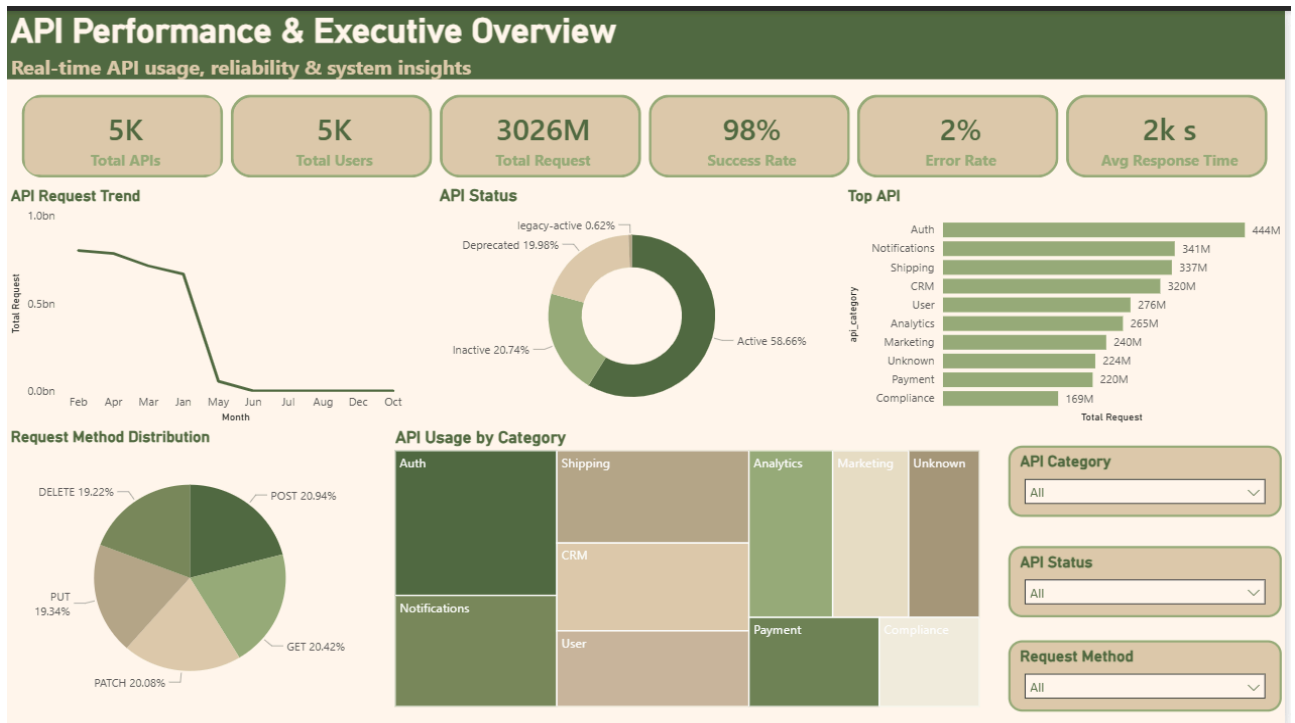
-- Response Time Ranking by Region

```
SELECT server_region,api_id,  
       AVG(response_time_ms) AS avg_response_time,  
       DENSE_RANK() OVER(  
         PARTITION BY server_region ORDER BY AVG(response_time_ms) DESC  
       ) AS response_rank  
FROM api_logs  
GROUP BY server_region, api_id;
```

9. Power BI Dashboard Development

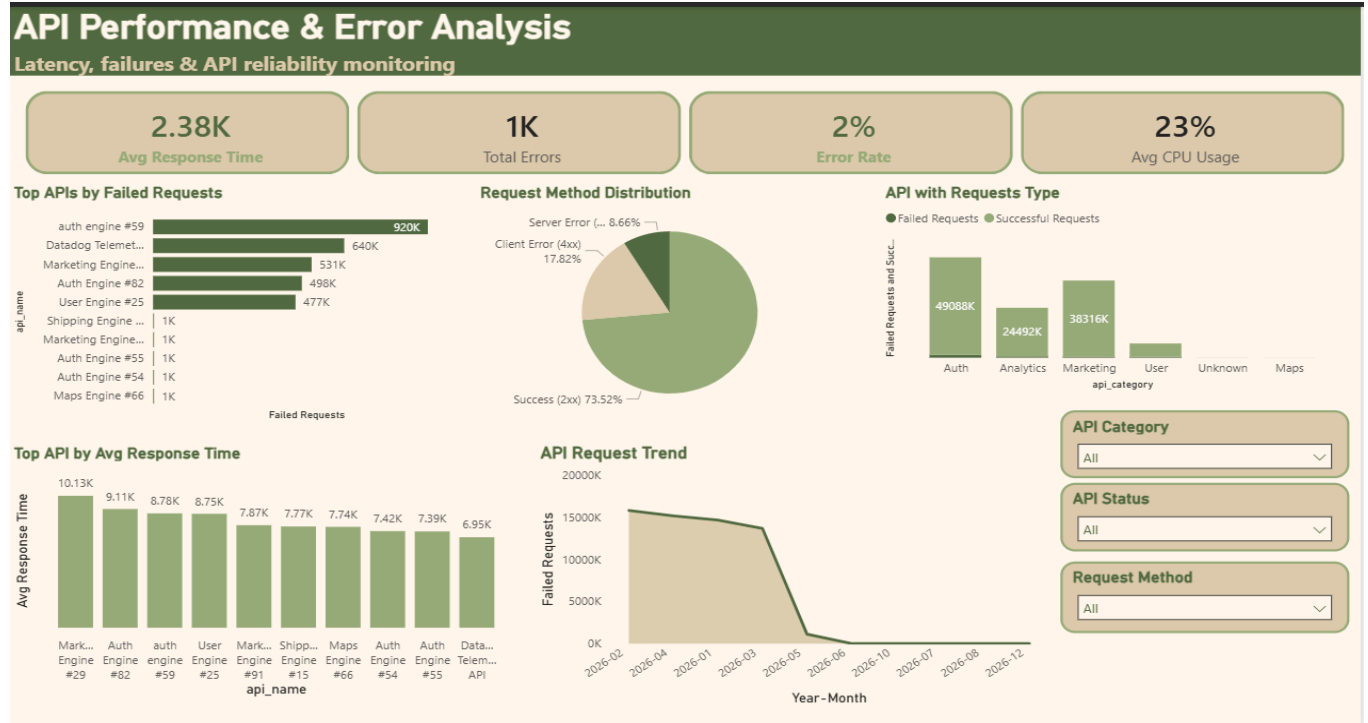
9.1 API Performance & Executive Overview Dashboard

- The dashboard gives a high-level overview of API ecosystem performance, usage, reliability, and request behavior.
- KPI cards show 5K APIs, 5K users, 3026M total requests, 98% success rate, 2% error rate, and 2.38K average response time.
- The API request trend chart shows request volume was initially very high and then dropped sharply after May, indicating reduced traffic or incomplete data collection.
- The API status donut chart highlights that most APIs are active, while inactive and deprecated APIs still represent a significant portion.
- The Top API bar chart shows Authentication, Notifications, and Shipping APIs generate the highest request traffic.
- The treemap and request method distribution show API usage is distributed across multiple categories and HTTP methods fairly evenly.



9.2 API Performance & Error Analysis Dashboard

- This dashboard focuses on API failures, latency, error monitoring, and operational reliability.
- KPI cards display 2.38K average response time, 1K total errors, 2% error rate, and 22.54 average CPU usage.
- The failed request chart identifies APIs with the highest number of failures.
- The request method distribution pie chart shows most requests are successful while client and server errors form a smaller portion.
- The response time chart compares APIs based on latency performance.
- The failed request trend line shows errors sharply declined after May, indicating either system improvements or incomplete records.

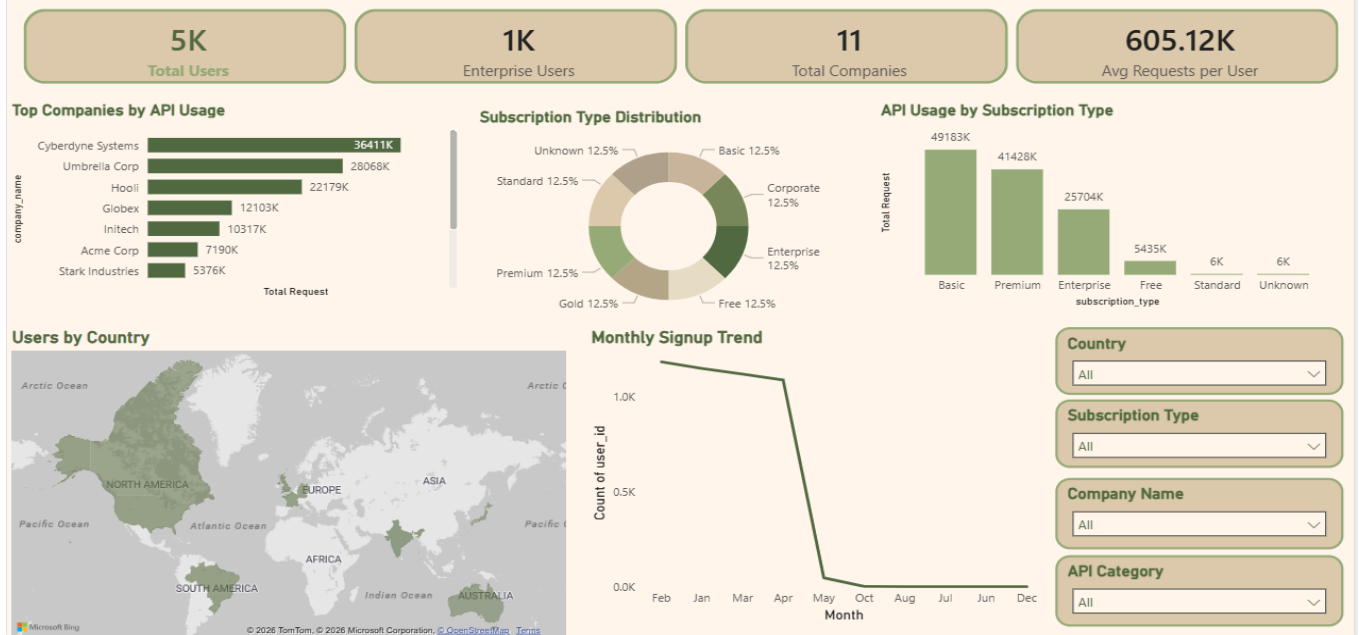


9.3 User & Usage Analytics Dashboard

- This dashboard analyzes user behavior, subscription patterns, and API consumption trends.
- KPI cards display 5K total users, 1K enterprise users, 11 companies, and 605.12K average requests per user.
- The company usage chart highlights organizations generating the highest API traffic.
- Subscription distribution shows API usage spread across Basic, Premium, Enterprise, and other plans.
- The global map visual displays API usage across different countries and regions.
- The monthly signup trend indicates user registrations sharply declined after April or May.

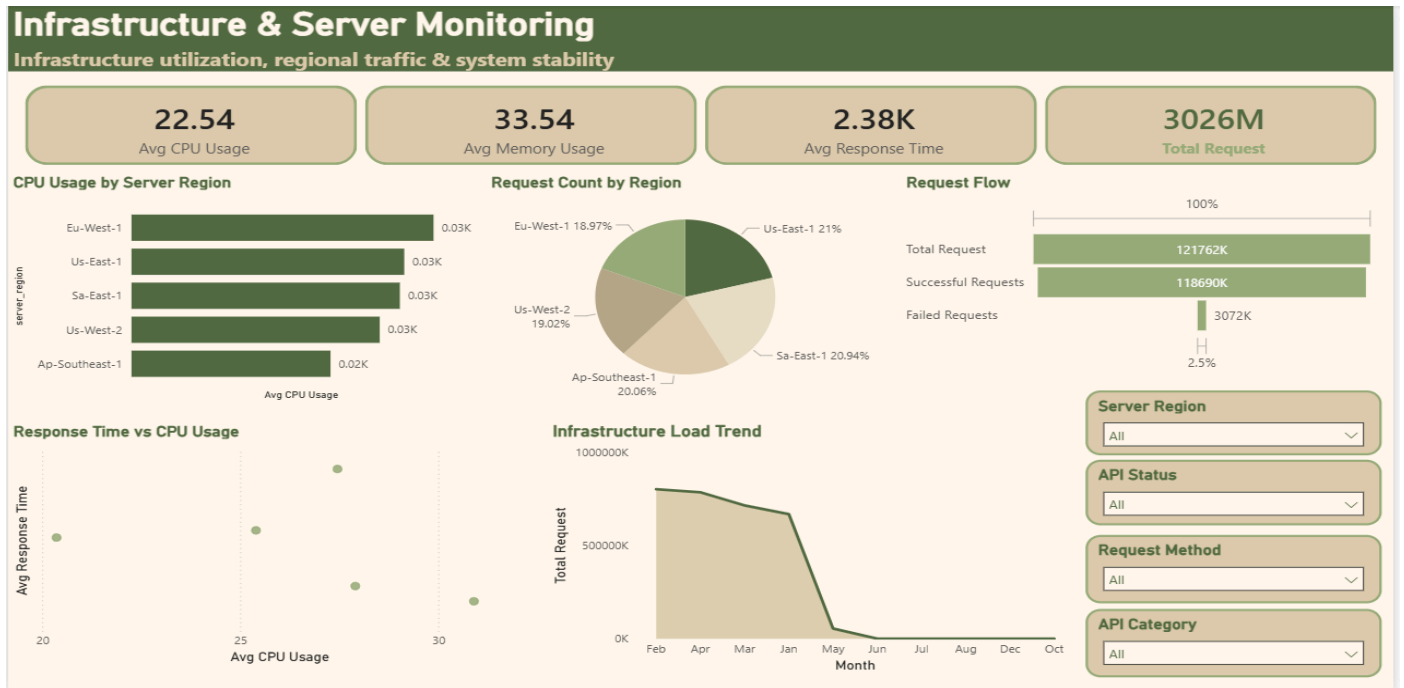
User & Usage Analytics

Customer behavior, subscriptions & API consumption insights



9.4 Infrastructure & Server Monitoring Dashboard

- This dashboard focuses on infrastructure performance, regional server activity, and system stability.
- KPI cards show 22.54 average CPU usage, 33.54 average memory usage, 2.38K average response time, and 3026M total requests.
- CPU usage by server region highlights resource utilization across multiple cloud regions.
- The request count distribution chart shows traffic is fairly balanced among regions.
- Request flow visualization compares successful versus failed requests.
- The infrastructure load trend reveals traffic sharply declined after May, similar to other dashboards.



10. Key Insights & Findings

1. The "May Cliff" Collapse

Traffic and new user signups abruptly plummeted to near-zero in May 2026 and never recovered, indicating a catastrophic, unresolved system failure.

2. Severe Latency

Average response times are critically slow ($\$2,000\text{\textit{s}}+$), driven primarily by the Auth and Marketing engines exceeding $\$8,000\text{\textit{s}}$.

3. Auth & Telemetry Risks

- Auth: The most heavily used service (Auth at 444M requests) is also the top source of failures (920K).
- Telemetry: 224M requests are untracked and labeled "Unknown", creating a major security blind spot.

4. Revenue Disconnect

Low-tier Basic and Premium users drive almost all platform traffic, while high-value Enterprise clients are barely using the product.

11. Business Recommendations

- **Deploy an Incident Forensic Team for the "May Cliff":** Treat the post-May traffic drop as a P0 catastrophic event. Determine if a missing database index, expired SSL certificate, broken billing gateway, or routing black hole caused the traffic and signup engine to freeze.
- **Overhaul Authentication & Gateway Architecture:** Decouple auth engine #59. Implement aggressive caching (e.g., Redis for session tokens), token-based authentication (JWTs) with longer TTLs, and rate-limiting to protect the auth infrastructure from being choked by high traffic volumes.
- **Enforce Strict API Route Telemetry:** Mandate that all engineering teams resolve the "Unknown" API category. Every endpoint must map to an owner and an official microservice category to prevent unmonitored shadow APIs from creating security holes.
- **Re-align Enterprise Value Propositions:** Contact the 1K Enterprise Users and 11 Total Companies immediately. They represent premium revenue but minimal API engagement. Conduct user discovery to identify why they aren't integrating the APIs into their workflows.

12. Conclusion

The dashboard ecosystem paints a picture of a high-potential API platform suffering from severe operational and architectural distress. While infrastructure capacity (CPU/Memory) is not a bottleneck, the software layer is crippled by extreme latency and a catastrophic drop-off in traffic after May. By stabilizing the core authentication services, fixing long-running synchronous requests, and investigating the root cause of the May system collapse, the platform can recover its traffic volume and better serve its high-value Enterprise clientele.

13. Future Improvements

- Implement Asynchronous Processing: For long-running endpoints (like the Marketing and Auth engines currently averaging >8,000 seconds), shift from synchronous REST calls to an asynchronous, message-driven architecture using tools like Apache Kafka or RabbitMQ. Return an immediate 202 Accepted status code and deliver data via webhooks.
- Establish Automated Circuit Breakers: Introduce a service mesh (e.g., Istio) with circuit-breaking capabilities. If auth engine #59 hits a 2% error threshold, fails fast and redirects traffic to a fallback container rather than letting requests linger for thousands of seconds.